The easiest way to understand the `passThrough` is along with `flexiFlow`. You use a `flexiFlow` when you need to process something after you commit it to Kafka. The `passThrough` is what you keep after the commit—what passes to the next stage.

The `plainSink` does nothing after the `flexiFlow`. Let's take another look at it:

```
package akka.kafka.scaladsl

object Producer {
  ...

  def plainSink[K, V](settings: ProducerSettings[K, V]):
           Sink[ProducerRecord[K, V], Future[Done]] =
    Flow[ProducerRecord[K, V]]
      .map(Message(_, NotUsed))
      .via(flexiFlow(settings))
      .toMat(Sink.ignore)(Keep.right)
  ...
}
```

It ignores the `passThrough` by setting `Sink.ignore` and `Keep.right`. It keeps the `Future[Done]` to return when the stream is finished.

With that understanding, let's look at the following example that reads from a topic, wraps each message in an `Envelope`, and commits it with a `committableSink`:

```
object ConsumerProducerAtLeastOnce {
  ...

      Consumer
      .committableSource(consumerSettings, Subscriptions.topics("test3"))
      .map { msg: CommittableMessage[String, String] =>
        ProducerMessage.single(                                    ❶
          new ProducerRecord[String, String](
```